Evo: Autoregressive–Diffusion Large Language Models with Evolving Balance

Junde Wu[1,2] **Minhao Hu**[1] **Jiayuan Zhu**[1] **Yuyuan Liu**[1]
Tianyi Zhang[2] **Jingkun Chen**[1] **Jiazhen Pan**[3]
Qirong Ho[4] **Min Xu**[5] **Yueming Jin**[2,*]
[1]University of Oxford [2]National University of Singapore [3]Stanford University
[4]MBZUAI [5]Carnegie Mellon University
[*]Corresponding author.

Abstract

We introduce **Evo**, a duality latent trajectory model that bridges autoregressive (AR) and diffusion-based language generation in a continuous framework. Evo represents generation as a latent flow: each token embedding evolves over a progression variable $t_i \in [0, 1]$ that indicates semantic maturity. Low values yield confident AR-like refinement, while high values invoke diffusion-style planning, enabling an uncertainty-adaptive balance. We show that AR and diffusion models emerge as discretizations of a shared probability flow and derive Evo's objective from a unified variational ELBO. Evo uses a time-conditioned Transformer governed by a shared vector field and is trained end-to-end to infer latent codes and progression times. During decoding, it performs semantics-aware refinement without sacrificing speed. Evo 8B achieves state-of-the-art or competitive results on 15 benchmarks spanning reasoning (GSM8K, ARC-C), code generation (HumanEval, MBPP), and general language understanding, while maintaining fast inference. These results establish a unified LLM design with strong generation quality, symbolic reasoning, and decoding efficiency.

1 Introduction

Large Language Models (LLMs) have emerged as a transformative paradigm in natural language processing, demonstrating remarkable capabilities in open-ended generation and reasoning (Brown et al., 2020; Wei et al., 2022). From the GPT series to instruction-tuned models, the predominant architecture driving these successes is the autoregressive (AR) Transformer, which models the conditional distribution of a sequence token-by-token in a left-to-right manner (Radford et al., 2018, 2019). This causal decoding framework has enabled scalable pretraining, fine-tuned alignment, and effective sampling. Yet, despite their ubiquity, AR-based LLMs operate in a strictly unidirectional generation regime and can exhibit compounding errors due to greedy or approximate decoding heuristics.

To address these challenges, a growing body of work has explored diffusion-based LLMs, generative models that synthesize text by iteratively denoising corrupted inputs (Gong et al., 2023; Sahoo et al., 2024; Zhu et al., 2026). Inspired by the success of diffusion models in continuous domains (Ho et al., 2020; Song et al., 2021), diffusion models for language replace one-pass AR decoding with a multi-step process that allows non-sequential token generation. This formulation enables iterative correction and improved global coordination. However, diffusion language models often require many inference steps, offer limited explicit control over high-level semantics, and can underperform AR models in perplexity.

In this paper, we introduce Evo, a duality latent flow model that generalizes autoregressive (AR) and diffusion-based language generation within a single, continuous generative framework. Unlike existing AR–diffusion hybrids, we treat autoregressive and diffusion generation as two manifestations of the same underlying diffusion process at different progression times in latent space. This inherent duality directly motivates Evo's design, allowing it to adaptively modulate the balance between AR-style refinement and diffusion-style planning based on the desired level of generation granularity, thereby jointly improving semantic clarity and detail richness. An illustration is shown in Figure 1.

Specifically, at the core of our framework lies a theoretical unification of AR and diffusion. We show that both emerge as discretizations of a shared probabilistic flow. We formalize this connection by defining latent trajectories governed by a time-indexed vector field and demonstrate

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, pages 3108–3121
October 24-29, 2026 ©2026 Association for Computational Linguistics

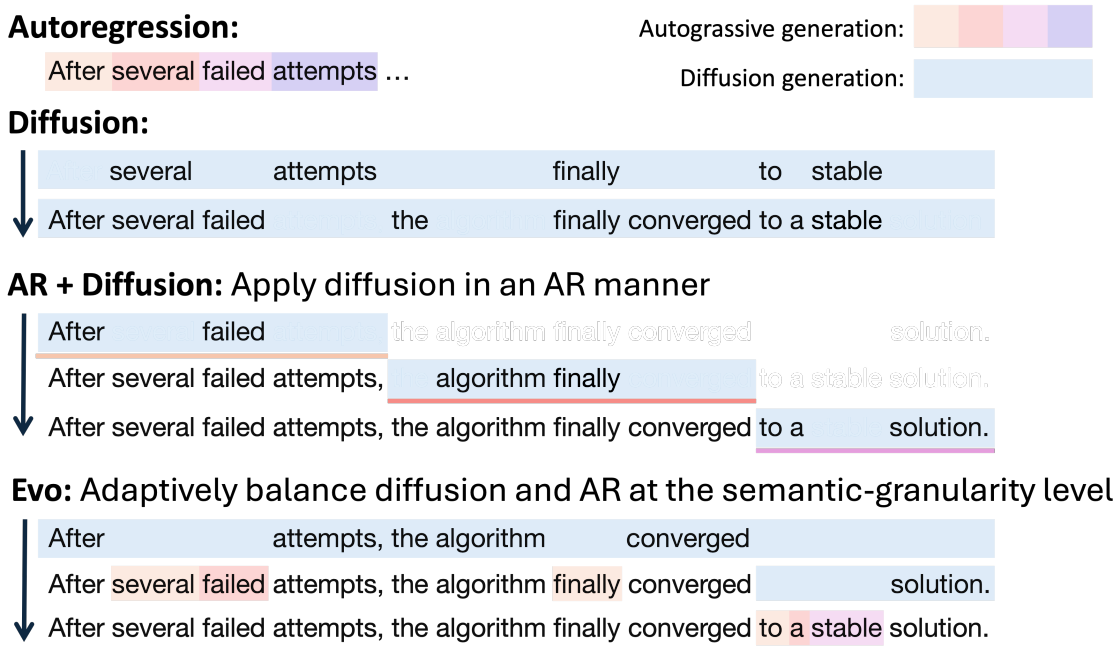


Figure 1: Evo generates text by evolving a diffusion-based semantic scaffold into token-level autoregressive realizations along a shared latent trajectory, allowing different tokens to reach semantic maturity at different times. Unlike prior autoregressive, diffusion, and AR+diffusion models, which either generate strictly left-to-right or apply diffusion in a fixed autoregressive order, Evo adaptively balances diffusion-style global planning and autoregressive next-token prediction by semantic granularity, selectively refining mature tokens while continuing to finalize the remaining scaffold within the same sentence.

that AR generation corresponds to deterministic flows near the origin, while diffusion-based decoding arises as stochastic score-following in the limit. This continuous perspective allows Evo to model generation as a structured path over latent states, with token-level progression times t_i that control how far each representation lies along the semantic continuum from planning to realization. Built on this foundation, Evo consists of a time-conditioned latent generator and a shared vector field $\mathcal{F}_\theta$ that governs semantic evolution. The model is trained via a variational objective that infers both latent embeddings and their associated time steps, optimizing an evidence lower bound (ELBO) on the sequence likelihood. The resulting training regime unifies next-token prediction and score-based denoising within a single differentiable loss, enabling end-to-end learning. As a special case, we show that Evo recovers both standard AR modeling and diffusion objectives and converges toward maximum likelihood estimation under mild regularity assumptions.

Empirically, Evo 8B achieves state-of-the-art or highly competitive results on 15 diverse benchmarks, including reasoning (GSM8K, ARC-C), code generation (HumanEval, MBPP), and general language understanding, while maintaining fast inference speed.

2 Approach

2.1 AR-Diffusion duality: Two Sides of the Same Coin from an evolving perspective

We propose a unified framework for explicit generative modeling, in which both autoregressive models and diffusion models are interpreted as structured paths over latent states governed by probabilistic transitions.

Formally, let $\mathbf{x} \in \mathbb{R}^d$ be a data sample drawn from the unknown distribution $p_{\text{data}}(\mathbf{x})$. We define a latent Markov path $\mathbf{z}^{(0)}, \mathbf{z}^{(1)}, \ldots, \mathbf{z}^{(L)}$ such that $\mathbf{z}^{(L)} = \mathbf{x}$, and posit that the generative model decomposes as

$$p_\theta(\mathbf{z}^{(0:L)}) = p_0(\mathbf{z}^{(0)}) \prod_{\ell=1}^{L} p_\theta(\mathbf{z}^{(\ell)} \mid \mathbf{z}^{(\ell-1)}), \quad (1)$$

where p_0 is a known initialization distribution (e.g., standard Gaussian), and p_θ parameterizes each transition. The interpretation of the intermediate states $\{\mathbf{z}^{(\ell)}\}$ and the design of transitions determine whether the resulting model behaves like an autoregressive generator or a diffusion sampler.

In the case of autoregressive models, the intermediate states correspond to partial completions

of the sequence. For a sequence $\mathbf{x} = (x_1, \ldots, x_T)$, we set $\mathbf{z}^{(t)} = (x_1, \ldots, x_t)$ and recover the familiar left-to-right factorization

$$p_\theta(\mathbf{x}) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}), \tag{2}$$

which corresponds to a deterministic, monotonic path in the space of prefix sequences. This process can be lifted into a continuous-time representation by interpolating the discrete sequence trajectory into a smooth path governed by an ODE. Specifically, we define a function $\mathbf{x}_s$ over $s \in [0, 1]$, where $\mathbf{x}_0 = \mathbf{0}$ and $\mathbf{x}_1 = \mathbf{x}$, and assume it evolves according to

$$\frac{d\mathbf{x}_s}{ds} = \mathcal{F}_\theta(\mathbf{x}_s, s), \tag{3}$$

with $\mathcal{F}_\theta$ being a parameterized vector field that controls how information is revealed along the generation path. This framing renders autoregression as a flow through latent space, aligning with recent formulations of continuous normalizing flows and neural ODEs.

In the case of diffusion models, recall that the forward transition is defined as

$$q(\mathbf{z}^{(t)} \mid \mathbf{z}^{(t-1)}) = \mathcal{N}(\sqrt{\alpha_t}\mathbf{z}^{(t-1)}, (1 - \alpha_t)\mathbf{I}), \tag{4}$$

with a fixed variance schedule $\beta_t = 1 - \alpha_t$. This forward process admits an analytic marginal $q(\mathbf{z}^{(t)} \mid \mathbf{x})$, enabling training of a reverse model $p_\theta(\mathbf{z}^{(t-1)} \mid \mathbf{z}^{(t)})$ that approximates the denoising transition. When discretized with sufficiently small noise steps, the forward process converges to an SDE

$$d\mathbf{z}_t = \mu(t)\, \mathbf{z}_t\, dt + \sigma(t)\, d\mathbf{w}_t, \tag{5}$$

and the corresponding reverse-time dynamics satisfy

$$d\mathbf{z}_t = \left[-\sigma(t)^2 \nabla_{\mathbf{z}} \log p_t(\mathbf{z})\right] dt + \sigma(t) d\bar{\mathbf{w}}_t, \tag{6}$$

where p_t is the time-marginal density and $d\bar{\mathbf{w}}_t$ is a reverse-time Brownian motion. Removing the stochastic term yields the deterministic probability flow ODE

$$\frac{d\mathbf{z}_t}{dt} = -\sigma(t)^2 \nabla_{\mathbf{z}} \log p_t(\mathbf{z}), \tag{7}$$

which is structurally identical to the AR flow in Eq. (3) under a reparameterization of time.

To make this equivalence precise, we define a smooth transformation $s(t) = \int_0^t \sigma(\tau)^2 d\tau$, mapping diffusion time $t \in [0, 1]$ to autoregressive scale $s \in [0, 1]$. Under this change of variables, both flows trace the same curves in $\mathbb{R}^d$ when the vector field $\mathcal{F}_\theta(\mathbf{x}, s)$ is chosen as the score function $-\nabla_{\mathbf{x}} \log p_s(\mathbf{x})$. This leads to a formal correspondence between autoregressive generation and score-based sampling, differing only in parametrization and directionality.

Considering the optimization procedures of both models. Autoregressive models directly optimize the negative log-likelihood (NLL), while diffusion models minimize a variational lower bound (ELBO) derived from the KL divergence between the forward and reverse paths. The gap between the ELBO and true NLL is given by

$$\log p_\theta(\mathbf{x}) - \mathcal{L}_{\text{ELBO}} = \sum_{t=1}^{T} \mathbb{E}_q \Big[\mathrm{KL}\big(q(\mathbf{z}^{(t-1)} \mid \mathbf{z}^{(t)}, \mathbf{x}) \,\|\, p_\theta(\mathbf{z}^{(t-1)} \mid \mathbf{z}^{(t)})\big)\Big]. \tag{8}$$

which vanishes as $\beta_t = \mathcal{O}(1/T)$ and $T \to \infty$, implying that $\mathcal{L}_{\text{ELBO}} \to \mathcal{L}_{\text{AR}}$.

According to the above analysis, autoregressive and diffusion models share the same underlying principle: both generate data by following a score-guided path through latent space. They differ mainly in how this path is parameterized– AR takes a deterministic, step-by-step route from data to data, while diffusion follows a stochastic reverse path from noise to data. Despite these differences, both can be viewed as Markovian approximations of the same continuous generative process. Since AR and diffusion can be seen as two ends of a continuous spectrum of path-based generative models, where AR excels at deterministic fine-grained refinement and diffusion at stochastic semantic planning, this unified perspective naturally motivates our hybrid design, assigning each model to the stage of generation where it performs best.

2.2 Progressive Latent Trajectory Modeling

We propose **Evo**, a duality latent flow model that generates text by evolving a sequence of latent vectors along a continuous semantic trajectory. Let $X = (x_1, \ldots, x_N) \in \mathcal{V}^N$ denote the target token sequence, and let $Z = \{z_i\}_{i=1}^N$, with each $z_i \in \mathbb{R}^d$ representing a latent code for token x_i. Genera-

tion is modeled as a learned vector field over the latent space, conditioned on a progression variable $t_i \in [0, 1]$ at each position. The variable t_i encodes the semantic maturity of token x_i, interpolating between coarse planning ($t_i \approx 1$) and fine-grained realization ($t_i \approx 0$).

We define the generation likelihood via a continuous-time latent flow:

$$p_\Theta(X) = \int p(X \mid Z)\, p_\theta(Z \mid \{t_i\})\, dZ, \quad (9)$$

where $p_\theta(Z \mid \{t_i\})$ is parameterized by a time-conditioned Transformer that evolves each latent vector z_i along its own trajectory:

$$\frac{dz_i(t)}{dt} = \mathcal{F}_\theta(z_i(t), t_i), \quad (10)$$

with $\mathcal{F}_\theta$ denoting the learned flow field shared across tokens and time. Each $z_i(0)$ is initialized from a prompt or prior distribution, and $z_i(1)$ is mapped to output token x_i via a decoding projection.

The per-token time-step t_i is predicted jointly with the latent vectors and controls both the generation behavior and its computational depth. In practice, we discretize t_i into K refinement steps, with smaller t_i requiring fewer updates and larger t_i enabling more extensive refinement. This enables adaptive, token-wise allocation of generation effort and supports efficient decoding with fidelity-aware early stopping.

By continuously parameterizing both the generation dynamics and the delegation from planning to refinement via t_i, Evo eliminates the need for discrete scaffolds or hard stage transitions. The result is a fully differentiable, flow-based generation process that smoothly integrates semantic intent and linguistic precision within a single unified model.

2.3 Training Objective

Evo is trained to maximize the log-likelihood of sequences $X \sim p^*$ under a continuous latent trajectory model. Since exact marginalization over latent paths $Z = \{z_i\}_{i=1}^N$ and progression variables $\{t_i\}_{i=1}^N$ is intractable, we optimize a variational lower bound:

$$\log p_\Theta(X) \geq \mathbb{E}_{Z,\, t \sim q(Z,t|X)} \Big[\log p_\theta(Z, t) + \log p(X \mid Z) - \log q(Z, t \mid X) \Big]. \quad (11)$$

where $q(Z, t \mid X)$ is an approximate posterior that infers both latent vectors and their associated generation times. The model $p_\theta(Z, t)$ defines a time-conditioned generative flow, while $p(X \mid Z)$ decodes the final latent states into tokens.

At each position i, the model predicts a latent vector z_i evolving along a learned trajectory with time-step $t_i \in [0, 1]$. We discretize the trajectory into K steps and train the model to predict the next latent state under the learned vector field $\mathcal{F}_\theta$:

$$\mathcal{L}_{\text{Flow}}(\theta) = \sum_{t=0}^{K-1} \mathbb{E}_{\substack{X \sim p^* \\ Z,\, t \sim q(Z,t|X)}} \Big[-\log p_\theta \Big(z^{(t+1)} \mid z^{(t)}, t \Big) \Big]. \quad (12)$$

where $z^{(0)}$ is initialized from the prompt or prior, and the flow steps are truncated for each token based on t_i. This loss generalizes both next-token prediction (at $t_i \approx 0$) and diffusion denoising (at $t_i \approx 1$), with all intermediate behaviors supported via the continuous time signal. We parameterize t_i as a continuous, learnable variable that reflects the confidence or semantic maturity of the latent vector z_i. The posterior $q(Z, t \mid X)$ jointly predicts z_i and t_i from the ground-truth sequence. During training, we estimate t_i via backpropagation to optimize the full ELBO.

We optionally initialize Evo from pretrained language models by mapping their token embeddings to latent initializations. Training can proceed end-to-end over both latent flows and time-step predictors, or with frozen early-stage priors. The model supports parameter sharing across all positions and times, enabling unified modeling of semantic planning and linguistic realization in a single trajectory-based framework.

2.4 Implementation Details

Architecture. Evo is implemented as a single Transformer-based sequence model with shared parameters across the entire generation trajectory. Each token position is associated with a continuous latent vector $z_i \in \mathbb{R}^d$ and a learned progression time-step $t_i \in [0, 1]$. The model operates in a time-conditioned fashion: at each refinement step, it receives $\{z_i^{(t)}, t_i\}$ as input and updates the latent trajectory using a learned vector field $\mathcal{F}_\theta$. The Transformer incorporates t_i into the computation via sinusoidal time embeddings added to each token's hidden state, allowing the model to modulate behavior across the coarse-

to-fine spectrum. Prompts are encoded as standard embeddings prepended to the sequence. We adopt a decoder-only architecture augmented to support full-sequence latent flow updates instead of purely causal decoding. Positional encodings and cross-token dependencies are preserved across time steps via full self-attention. The model supports flexible refinement schedules and can interpolate seamlessly between autoregressive and denoising behavior via the learned t_i values.

Refinement Schedule. Generation proceeds over a fixed number of refinement steps $K_{\max}$ (typically 20), where each latent vector z_i is updated according to its learned time-step t_i. Rather than masking tokens or injecting discrete corruption, we use continuous noise perturbation during training. Specifically, we add Gaussian noise to the latent embeddings proportional to a scheduled diffusion time t, and train the model to denoise and reconstruct the true target embeddings. At each step t, the update rule follows a discretized ODE formulation:

$$z_i^{(t+1)} = z_i^{(t)} + \Delta t \cdot \mathcal{F}_\theta(z_i^{(t)}, t_i),$$

where Δt is the step size and $\mathcal{F}_\theta$ is the shared time-conditioned flow field. For tokens with lower t_i, updates converge quickly; for higher t_i, refinement proceeds across more steps. In practice, we optionally truncate updates per token after $K_i = \lfloor K_{\max} \cdot t_i \rfloor$ to save computation, though full trajectories remain supported for all tokens.

Sampling. During inference, the model first samples initial latent states $z_i^{(0)}$ and progression steps t_i for each token using top-p nucleus sampling ($p = 0.9$, temperature 0.7) over the latent prior distribution. These samples define the initial semantic sketch and generation depth per token. The model then refines the latent trajectory deterministically over $K_{\max}$ steps without adding noise, producing stable and coherent generations. The final token sequence is decoded via a linear projection or nearest neighbor search over a pretrained embedding space.

3 Experiments

3.1 Experimental Setting

Training Setup. We train **Evo** at five parameter scales from 1.3B to 13B, and report all main results using the XL configuration (8.5B). Model depth and width are scaled proportionally across sizes. Evo is a decoder-only Transformer augmented with a continuous progression variable $t_i \in [0, 1]$ and a shared latent flow with full-sequence self-attention. Pretraining is conducted on a 3.4T-token multi-domain corpus with a maximum context length of 4K tokens, using a 64K SentencePiece BPE tokenizer. Optimization follows standard large-scale LLM training with AdamW, cosine learning rate scheduling, bfloat16 mixed precision, and DeepSpeed ZeRO-3 on NVIDIA A100 GPUs. After pretraining, Evo is supervised fine-tuned using the same post-training dataset mixture and training settings as LLaDA (Zhu et al., 2026). Further details are provided in the Appendix.

Baselines, and Evaluation. We evaluate Evo 8B against recent open-source models of comparable scale, including AR-only (LLaMA3 8B (Grattafiori et al., 2024), Qwen2.5 7B (Hui et al., 2024)), diffusion-only (LLaDA 8B (Zhu et al., 2026), MDLM 7B (Sahoo et al., 2024)), and AR+diffusion hybrids (BD3-LM 7B (Arriola et al., 2025), ARD 7B (Wu et al., 2023)). The benchmark suite covers general understanding and reasoning, mathematics and science, code generation, and Chinese language understanding. For LLaMA3 and Qwen2.5, we initialize from their publicly released pretrained checkpoints and apply the same post-training procedure. Other diffusion-based and hybrid baselines are trained following their respective papers and code, with training data sources, tokenization, and evaluation protocols matched to our setup. All models are evaluated under identical prompts, in-context learning settings, and decoding configurations. Code tasks use deterministic decoding (temperature 0) and pass@1 accuracy, while reasoning and knowledge tasks report final-answer accuracy. Inference speed is measured in tokens per second on a single NVIDIA A100 GPU, including prompt encoding time. Further details are provided in the Appendix.

3.2 Main Results

We present our results in Tab. 1 (pre-trained) and Tab. 2 (post-trained), together with end-to-end latency and decoding throughput measured under a unified single-request inference setting.

Performance. Evo 8B achieves the strongest and most balanced performance across all capability categories, with its largest margins appear-

Table 1: Benchmark Results of Pre-trained LLMs. Evo 8B, BD3-LM 7B, ARD 7B (AR + Diffusion hybrid), LLaDA 8B, MDLM 7B (Diffusion-based), LLaMA3 8B, and Qwen2.5 7B (AR-based) are evaluated under the same protocol. The numbers in parentheses represent the number of shots.

Model	Evo 8B	BD3-LM 7B	ARD 7B	LLaDA 8B	MDLM 7B	LLaMA3 8B	Qwen2.5 7B
E2E Latency (s)	8.6	14.2	32.5	21.8	18.9	7.4	8.1
Inference Speed (tokens/s)	52	28	12	16	22	58	46
General Tasks							
MMLU	**76.8** (5)	70.0 (5)	46.2 (5)	65.6 (5)	64.0 (5)	65.1 (5)	74.5 (5)
BBH	68.4 (3)	61.8 (3)	36.9 (3)	49.6 (3)	55.8 (3)	57.9 (3)	**70.1** (3)
ARC-C	**65.6** (25)	60.2 (25)	46.7 (0)	48.2 (0)	59.6 (25)	52.8 (0)	64.0 (25)
HellaSwag	82.1 (0)	**83.0** (10)	75.6 (0)	72.2 (0)	81.2 (10)	78.8 (0)	80.5 (10)
TruthfulQA	**58.1** (0)	53.8 (0)	39.4 (0)	46.8 (0)	42.0 (0)	44.5 (0)	56.7 (0)
WinoGrande	76.3 (5)	**78.1** (5)	72.2 (5)	75.1 (5)	77.4 (5)	77.6 (5)	75.6 (5)
PIQA	**81.2** (0)	-	79.4 (0)	74.7 (0)	-	80.3 (0)	-
Mathematics & Science							
GSM8K	**86.4** (4)	79.8 (4)	15.1 (4)	70.9 (4)	36.5 (4)	52.7 (4)	85.8 (4)
Math	**54.9** (4)	42.9 (4)	3.6 (4)	27.0 (4)	10.0 (4)	15.4 (4)	50.2 (4)
GPQA	**38.4** (5)	30.5 (5)	25.3 (5)	26.4 (5)	24.9 (5)	26.2 (5)	36.1 (5)
Code							
HumanEval	**60.6** (0)	50.6 (0)	13.4 (0)	33.2 (0)	29.0 (0)	34.6 (0)	58.3 (0)
HumanEval-FIM	73.4 (2)	71.8 (0)	27.2 (2)	**74.0** (2)	-	73.0 (2)	-
MBPP	**77.4** (4)	63.7 (0)	18.9 (4)	38.5 (4)	51.3 (0)	47.1 (4)	75.2 (0)
Chinese							
CMMLU	**88.1** (5)	84.2 (5)	33.1 (5)	70.2 (5)	47.0 (5)	50.4 (5)	-
C-Eval	80.4 (5)	**82.7** (5)	34.6 (5)	70.1 (5)	45.4 (5)	52.0 (5)	-

ing on tasks that require explicit global planning followed by precise constraint satisfaction. In the pre-trained setting, Evo leads on GSM8K (86.4), MATH (54.9), GPQA (38.4), HumanEval (60.6), and MBPP (77.4), substantially outperforming AR-only baselines such as LLaMA3 on MATH (+24.0) and HumanEval (+25.0). These gains indicate that strictly sequential AR decoding struggles once early reasoning errors are committed. Compared to the strongest AR+diffusion hybrid baseline BD3-LM, Evo further improves GSM8K (+6.6), MATH (+12.0), GPQA (+7.9), and HumanEval (+10.0), suggesting that fixing the AR–diffusion boundary at the block level limits the model's ability to refine high-level reasoning decisions. After post-training, the same pattern persists: Evo reaches 92.5 on ARC-C, 78.8 on MATH, and 86.7 on HumanEval, yielding a more uniform capability profile than models that peak on isolated benchmarks (e.g., Qwen2.5 on GSM8K).

Efficiency. These performance gains are achieved without incurring the high inference cost typically associated with diffusion-based generation because Evo restricts diffusion-style computation to semantically uncertain regions rather than applying it uniformly across tokens or blocks. As a result, Evo operates at near–AR decoding speed (52 tokens/s, 8.6 s end-to-end latency), closely matching LLaMA3 (58 tokens/s, 7.4 s) and Qwen2.5 (46 tokens/s, 8.1 s), while substantially outperforming other AR–diffusion hybrids and diffusion-only models in latency. In contrast, BD3-LM performs mandatory diffusion refinement at the block level, introducing additional model evaluations even when large portions of the sequence are already semantically resolved; this rigidity manifests as higher latency (14.2 s). AR-Diffusion further amplifies this inefficiency by enforcing position-dependent denoising schedules that allocate the most refinement steps to later tokens regardless of semantic certainty, leading to both high latency (32.5 s). Diffusion-only models such as LLaDA and MDLM incur even larger latency penalties (21.8 s and 18.9 s, respectively) because iterative denoising is applied across the entire sequence.

Together, these results show that Evo's end-to-end duality, which learns to allocate diffusion-style global planning and AR-style local refinement along a shared decoding trajectory, enables early commitment to low-entropy structure where necessary and selective refinement only where uncertainty remains. This adaptive coordination directly translates into stronger reasoning and code performance and practical inference efficiency, improving the accuracy–efficiency frontier over both rigid hybrids and diffusion-dominant approaches.

Table 2: Benchmark Results of Post-trained LLMs. **Evo 8B** and LLaDA 8B post-trained via SFT. The numbers in parentheses represent the number of shots.

Model	Evo 8B	BD3-LM 7B	ARD 7B	LLaDA 8B	MDLM 9B	LLaMA3 8B	Qwen2.5 7B
E2E Latency (s)	8.6	14.2	32.5	21.8	18.9	7.4	8.1
Inference Speed (tokens/s)	52	28	12	16	22	58	46
Post-training	SFT	SFT+RL	SFT+RL	SFT	SFT+RL	SFT+RL	SFT+RL
General Tasks							
MMLU	**78.6** (5)	75.1 (5)	48.9 (5)	65.5 (5)	70.2 (5)	68.4 (5)	73.8 (5)
MMLU-pro	**57.2** (0)	48.1 (5)	12.8 (0)	37.0 (0)	52.1 (5)	41.9 (0)	56.3 (5)
Hellaswag	**86.4** (0)	84.8 (0)	50.2 (0)	74.6 (0)	-	75.5 (0)	82.2 (0)
ARC-C	**92.5** (0)	86.7 (0)	56.9 (0)	88.5 (0)	-	82.4 (0)	-
Mathematics & Science							
GSM8K	89.3 (4)	85.7 (0)	31.0 (4)	78.6 (4)	76.7 (0)	78.3 (4)	**91.6** (0)
Math	**78.8** (0)	74.3 (0)	15.6 (0)	26.6 (0)	44.3 (0)	29.6 (0)	75.5 (0)
GPQA	**39.1** (5)	34.3 (0)	24.2 (5)	31.8 (5)	32.8 (0)	31.9 (5)	36.4 (0)
Code							
HumanEval	**86.7** (0)	82.9 (0)	16.5 (0)	47.6 (0)	68.9 (0)	59.8 (0)	84.8 (0)
MBPP	**81.1** (4)	67.2 (0)	20.6 (4)	34.2 (4)	74.9 (0)	57.6 (4)	79.2 (0)

3.3 Scalability

We investigate the scaling properties of Evo by varying model size and refinement steps. We train 15 variants across five model sizes (S, B, L, XL, H) and three refinement schedules ($K \in \{5, 10, 20\}$), denoted as Evo-{size}/{steps}. Model sizes range from S (1.3B parameters, 24 layers, 1024d latents, 16 heads) to H (13B parameters, 48 layers, 2560d latents, 40 heads), with intermediate sizes B (2.7B), L (5.2B), and XL (8.5B) following proportional scaling of depth and width. Each base architecture is evaluated with 5, 10, and 20 refinement steps to study the interplay between model capacity and iterative refinement depth. This design spans computational budgets from around 10^{21} to 10^{23} FLOPs. In Fig. 2, we observe robust correlation observed between model Flops and performance. This implies that Evo has good scaling capability given more computations.

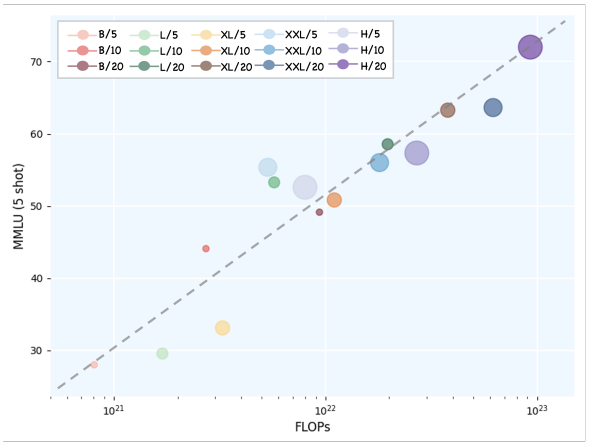


Figure 2: The scaling capability of Evo on MMLU.

3.4 Ablation Study

We conduct ablations to validate Evo's design, summarized in Figures 3 and 4. Evaluations cover perplexity (PPL), average task accuracy (MMLU, GSM8K), and refinement steps per token.

Effect of Time Conditioning. Removing time conditioning severely hurts PPL and accuracy, showing its role in modulating generation along a continuous trajectory. Fixed time-steps also underperform: $t_i = 0$ (pure AR) yields the worst accuracy, $t_i = 1$ (pure diffusion) improves fluency but is inefficient, and $t_i = 0.5$ gives a trade-off but remains inferior to learned time.

Learned vs. Heuristic Steps. A heuristic based on AR entropy performs reasonably but lags behind learned t_i, indicating that confidence signals alone are insufficient and direct end-to-end optimization is essential.

Architecture and Pretraining. Replacing Evo's unified latent flow with separate AR and diffusion modules lowers accuracy and raises PPL, highlighting the value of joint modeling. Removing shared embeddings weakens alignment across steps, and dropping pretrained initialization slows convergence and reduces accuracy. By contrast, freezing the AR backbone retains most performance, showing Evo's compatibility with modular adaptation.

Overall, Evo's gains arise from its time-conditioned latent flow, unified architecture, and shared representations.

Ablation Variant	PPL ↓	Task Acc ↑
w/o Time Conditioning	9.6	69.1
Fixed $t_i = 0$ *(AR-only)*	10.2	69.6
Fixed $t_i = 1$ *(Diff-only)*	8.6	77.0
Fixed $t_i = 0.5$ *(Uniform)*	8.8	77.9
Heuristic t_i *(entropy)*	8.4	78.2
Evo (full)	**8.1**	**81.6**

Figure 3: Ablation study of Evo on text modeling and reasoning tasks. PPL: perplexity on validation set; Task Acc: average task accuracy (MMLU and GSM8K). Results highlight the importance of time conditioning, learned token-wise t_i, and adaptive refinement.

Ablation Variant	PPL ↓	Task Acc ↑
Two-Module (AR + Diff)	8.6	77.9
No Shared Embeddings	8.4	78.3
No Pretrained Init	9.1	76.2
Frozen AR Backbone	8.7	80.4
Evo (full)	**8.1**	**81.6**

Figure 4: Ablation on architecture and pretraining. Evo benefits from a unified model, embedding sharing, and pretrained initialization. Task accuracy is averaged across MMLU and GSM8K.

4 Related Work

4.1 Autoregressive Language Models and Diffusion-Based Language Models

Autoregressive models generate text by factorizing the joint probability of a sequence into a product of conditional probabilities, typically producing tokens sequentially from left to right. Transformer-based architectures, such as GPT (Radford et al., 2018), GPT-2 (Radford et al., 2019), and subsequent large-scale models like GPT-3 (Brown et al., 2020), have demonstrated remarkable success in generating coherent and contextually relevant text. These models rely on causal masking to enforce sequential dependencies, enabling efficient training and sampling. However, AR models often struggle with global coherence over long sequences and lack mechanisms for iterative refinement

Diffusion models, originally popularized in image generation (Ho et al., 2020; Song et al., 2021), have recently been adapted for text generation. These models operate by iteratively denoising a noisy sequence, typically starting from a random noise distribution and refining it toward a target sequence. Works such as D3PM (Austin et al., 2021) and Diffusion-LM (Li et al., 2022) have explored discrete diffusion processes for text, modeling token distributions over a series of denoising steps. Continuous diffusion approaches, such as those proposed by Song et al. (2021), have also been extended to text by embedding tokens in a continuous space (Gong et al., 2023).

4.2 Autoregressive and Diffusion Hybrid Models

Recent work has explored combining the complementary strengths of autoregressive (AR) and diffusion-based generation for text, aiming to balance causal decoding with iterative refinement. Early hybrid attempts include step-unrolled denoising autoencoders (Savinov et al., 2022), which integrate denoising objectives into a sequential generation process, and flow-based categorical models (Hoogeboom et al., 2021), which bridge discrete token spaces and continuous latent dynamics. While these approaches demonstrate that diffusion-style refinement can be reconciled with sequential generation, they typically rely on separate training objectives or predefined inference schedules and do not explicitly model how global semantic structure should stabilize and be refined during generation.

More recent work has focused on tighter AR–diffusion hybrids with explicit causal structure. AR-Diffusion (ARD) (Wu et al., 2023) extends SSD-LM (Han et al., 2023) by imposing a left-to-right noise schedule, ensuring causal dependency while retaining diffusion-based denoising. Later, BD3-LM (Arriola et al., 2025) adopts a discrete block diffusion formulation, generating blocks autoregressively while applying diffusion-based refinement within each block. This strategy improves efficiency over fully masked diffusion but fixes the interaction between planning and refinement at the block level, constraining the model's ability to adapt refinement granularity dynamically within a sentence.

5 Conclusion

In this work, we introduced Evo, a unified latent flow model that bridges AR and diffusion paradigms through a continuous trajectory. Evo integrates confident planning and fine-grained refinement within a single architecture. Our theoretical analysis established AR and diffusion as discretizations of the same probability flow, and our empirical results demonstrated that Evo achieves state-of-the-art performance across diversebenchmarks while maintaining fast decoding.

6 Limitations

While Evo demonstrates strong performance and a favorable accuracy–efficiency trade-off, it also has several limitations that warrant discussion.

Training Cost. Evo introduces additional training complexity compared to standard autoregressive or diffusion-only models, as it must jointly learn to evolve a semantic scaffold and coordinate autoregressive commitment with diffusion-based refinement along a shared latent trajectory. Although we adopt efficient training strategies to amortize this cost, Evo training remains more expensive than pure AR training and moderately more costly than standard diffusion training. Further optimization of the training objective, curriculum, and scheduling may be required to reduce this overhead, particularly for larger model scales.

Inference Efficiency Sensitivity. Although Evo avoids the high inference cost typical of diffusion-based generation, its decoding efficiency depends on how semantic uncertainty is distributed across tokens. Tasks that require prolonged refinement over large portions of the sequence may reduce the efficiency advantage over AR-only models. In addition, while Evo mitigates the rigid scheduling constraints of prior AR–diffusion hybrids, the optimal balance between planning and refinement can still be task-dependent, and suboptimal allocation may affect either latency or output quality.

Scalability and Controllability. Evo generates text by evolving a shared semantic scaffold rather than committing strictly left-to-right, which introduces new challenges for controllability and interpretability. While this flexibility enables stronger reasoning, it may complicate fine-grained control over generation attributes such as length, especially in long-form generation. Understanding how to best guide or constrain scaffold evolution remains an open problem.

General Generative Model Risks. Like other large generative language models, Evo is subject to well-known limitations, including hallucination, exposure to training data biases, potential copyright infringement, and the risk of generating harmful or misleading content. Addressing these issues requires complementary techniques such as alignment training, filtering, and post-hoc safety mechanisms, which are orthogonal to the core modeling contributions of this work.

Overall, while Evo advances the integration of autoregressive and diffusion-based generation, these limitations highlight important directions for future research in training efficiency, inference control, and safe deployment.

References

Marianne Arriola, Aaron Kerem Gokaslan, Justin T. Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. 2025. Block diffusion: Interpolating between autoregressive and diffusion language models. In *The Thirteenth International Conference on Learning Representations*.

Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. 2021. Structured denoising diffusion models in discrete state-spaces. *Advances in Neural Information Processing Systems*, 34:17981–17993.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901.

Shansan Gong, Mukai Li, Jiangtao Feng, Zhiyong Wu, and Lingpeng Kong. 2023. DiffuSeq: Sequence to sequence text generation with diffusion models. In *The Eleventh International Conference on Learning Representations*.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Xiaochuang Han, Sachin Kumar, and Yulia Tsvetkov. 2023. SSD-LM: Semi-autoregressive simplex-based diffusion language model for text generation and modular control. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11575–11596, Toronto, Canada. Association for Computational Linguistics.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851.

Emiel Hoogeboom, Didrik Nielsen, Priyank Jaini, Patrick Forré, and Max Welling. 2021. Argmax flows and multinomial diffusion: Learning categorical distributions. *Advances in Neural Information Processing Systems*, 34:12454–12465.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang,

Bowen Yu, Keming Lu, et al. 2024. Qwen2.5-Coder technical report. *arXiv preprint arXiv:2409.12186*.

Xiang Lisa Li, John Thickstun, Ishaan Gulrajani, Percy Liang, and Tatsunori B. Hashimoto. 2022. Diffusion-LM improves controllable text generation. *Advances in Neural Information Processing Systems*, 35:4328–4343.

Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. 2018. Improving language understanding by generative pre-training. Technical report, OpenAI.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. Technical report, OpenAI.

Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T. Chiu, Alexander Rush, and Volodymyr Kuleshov. 2024. Simple and effective masked diffusion language models. *Advances in Neural Information Processing Systems*, 37:130136–130184.

Nikolay Savinov, Junyoung Chung, Mikolaj Binkowski, Erich Elsen, and Aaron van den Oord. 2022. Step-unrolled denoising autoencoders for text generation. In *The Tenth International Conference on Learning Representations*.

Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. 2021. Score-based generative modeling through stochastic differential equations. In *The Ninth International Conference on Learning Representations*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837.

Tong Wu, Zhihao Fan, Xiao Liu, Yeyun Gong, Yelong Shen, Jian Jiao, Hai-Tao Zheng, Juntao Li, Zhongyu Wei, Jian Guo, Nan Duan, and Weizhu Chen. 2023. AR-Diffusion: Auto-regressive diffusion model for text generation. *Advances in Neural Information Processing Systems*, 36:39957–39974.

Fengqi Zhu, Rongzhen Wang, Shen Nie, Xiaolu Zhang, Chunwei Wu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. 2026. LLaDA 1.5: Variance-reduced preference optimization for large language diffusion models. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11425–11460, San Diego, California, United States. Association for Computational Linguistics.

A Training Details

We trained Evo across five model scales, ranging from 1.3B to 13B parameters, with the XL configuration (8.5B parameters) used for the main experiments. Architectural hyperparameters were scaled proportionally in depth and width to maintain consistent capacity-to-compute ratios. Evo extends a standard decoder-only Transformer by introducing a continuous progression time embedding $t_i \in [0, 1]$ that is injected into each token representation via a sinusoidal projection. A shared vector field F_θ is applied across all tokens and refinement steps, updating latent states in a time-conditioned manner. Latent flow updates replace purely causal masking while preserving inter-token dependencies through full-sequence self-attention. At every refinement step, the model attends over the entire latent sequence, modulating generation behaviour along the coarse-to-fine spectrum dictated by t_i.

The pretraining corpus consists of 3.4 trillion tokens drawn from a mixture of domains. English text, accounting for roughly sixty percent of the data, is sourced from filtered Common Crawl, Wikipedia, books, StackExchange, ArXiv, and code repositories. Multilingual data covers more than thirty languages with high-quality Wikipedia and curated web sources, contributing about twenty-five percent of the total. Code data comprises approximately ten percent, obtained from permissively licensed GitHub repositories in languages including Python, JavaScript, C/C++, Java, Go, and Rust. Mathematics and science resources, representing the remaining five percent, are drawn from datasets such as ProofNet, MathQA, and scientific publications. Data cleaning includes deduplication with MinHash, language identification filtering, toxicity removal, and heuristic screening for quality. All documents are normalized to a maximum length of 4K tokens for efficient batching.

Tokenization is performed using a SentencePiece BPE vocabulary of 64K merge operations, with case-sensitive encoding and special tokens for sequence control and model operations, including a dedicated token to explicitly condition on progression time. This facilitates training regimes where t_i can be externally specified or learned.

The training objective is a unified variational lower bound that combines three components: a latent flow prediction loss, a standard autoregressive negative log-likelihood term, and a score-

matching loss for denoising. The autoregressive term dominates when t_i is close to zero, while the score-matching term dominates as t_i approaches one. We set the weights of the autoregressive and diffusion losses to unity and anneal their relative importance during warmup, allowing the latent flow loss to govern later training. The optimizer is AdamW with $\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-8}$, and weight decay of 0.1. A cosine learning rate schedule with three thousand warmup steps is used, with peak learning rates of 2×10^{-4} for smaller models and 1.5×10^{-4} for larger ones. Batch sizes correspond to two million tokens for smaller models and four million for larger ones, with a fixed sequence length of 4K tokens and gradient accumulation employed to fit the model within available GPU memory.

Noise is injected into the latent space using a continuous Gaussian perturbation $z_i^{(t)} = \sqrt{\alpha_t} z_i^{(0)} + \sqrt{1 - \alpha_t}\, \epsilon$, where α_t follows a cosine schedule $\alpha_t = \cos^2\left(\frac{\pi}{2} t\right)$. This allows smooth interpolation between deterministic autoregressive behaviour and stochastic diffusion-like refinement. During inference, noise is disabled and generation proceeds deterministically along the learned ODE path.

Progression times t_i are predicted by a learned head applied to the initial latent vector of each token, consisting of a two-layer MLP with GELU activation and a final sigmoid mapping to $[0, 1]$. An entropy regularizer encourages diverse t_i usage across the sequence. The per-token update depth K_i is defined as $\lfloor K_{\max} \cdot t_i \rfloor$, with $K_{\max}$ set to 20 in pretraining. Tokens with smaller t_i values converge quickly with few updates, while larger values permit extended refinement. This mechanism enables adaptive allocation of computation to uncertain tokens.

Regularization techniques include dropout of 0.1 on residual and feed-forward connections, label smoothing of 0.1 for the autoregressive loss, gradient clipping at a global norm of 1.0, and exponential moving averages of model weights with decay 0.9999 to stabilize inference. Mixed-precision training is performed in bfloat16 for improved efficiency.

Pretraining is conducted on large-scale GPU clusters, with smaller models trained on sixty-four NVIDIA A100 40GB GPUs and larger ones on up to two hundred and fifty-six NVIDIA A100 80GB GPUs. DeepSpeed ZeRO-3 with pipeline parallelism and fully sharded data parallelism is used to minimize memory overhead. Sequence parallelism enables efficient handling of long contexts. Training wall-clock times range from two weeks for the smallest models to seven weeks for the largest.

Post-training consists of supervised fine-tuning on curated instruction-following datasets, reasoning corpora, and code generation benchmarks. The fine-tuning learning rate is set to 5×10^{-5} with cosine decay and one thousand warmup steps. Batch sizes are set to 512 sequences of 2K tokens, and training runs for 30K–50K steps depending on the domain. For alignment, we experiment with direct preference optimization for helpfulness and harmlessness, as well as self-refinement loops in which the model generates candidate completions, scores them with an internal reward model, and refines outputs via additional deterministic refinement steps.

Evaluation is performed under identical in-context learning settings for all baselines, with both zero-shot and few-shot prompts, as well as chain-of-thought prompting where appropriate. For code benchmarks, we report pass@1 scores using deterministic decoding with temperature zero. Reasoning tasks are evaluated based on accuracy of final answers, and inference speed is measured in tokens per second on a single A100 GPU, including prompt encoding time. This consistent setup ensures fair comparison across autoregressive, diffusion-based, and hybrid architectures, allowing us to isolate the benefits of Evo's time-conditioned latent flow design.

B Proof of Proposition

Proposition 1. *Autoregressive language models and score-based diffusion language models are two discrete parameterizations of the same continuous-time generative process. Specifically, there exists a smooth, monotonic reparameterization of time $s(t)$ such that the deterministic probability flow ODE of a diffusion model coincides with the latent evolution ODE of an autoregressive model, when both are expressed in a continuous latent space.*

Proof. We begin by recalling the continuous-time latent evolution for an autoregressive (AR) model. Let x_s denote the latent representation of the partially generated sequence at continuous generation time $s \in [0, 1]$, with x_0 denoting the start state

and x_1 the completed sequence. In the continuous limit of left-to-right generation, the dynamics can be written as an ordinary differential equation

$$\frac{dx_s}{ds} = F_\theta(x_s, s), \tag{13}$$

where F_θ is a deterministic vector field representing the model's generative policy over the latent space.

Now consider a continuous-time score-based diffusion model. Let z_t denote the latent variable at diffusion time $t \in [0, 1]$, where $z_1 \sim \mathcal{N}(0, I)$ is pure noise and z_0 is the data distribution. The forward noising process can be described by a variance-preserving stochastic differential equation (SDE):

$$dz_t = \mu(t) z_t \, dt + \sigma(t) \, dw_t, \tag{14}$$

where w_t is a standard Wiener process. The corresponding reverse-time SDE is given by:

$$\begin{aligned} dz_t = &\left[-\mu(t) z_t - \sigma(t)^2 \nabla_z \log p_t(z_t)\right] dt \\ &+ \sigma(t) \, d\bar{w}_t, \end{aligned} \tag{15}$$

where $\bar{w}_t$ is reverse-time Brownian motion and p_t denotes the marginal distribution of z_t.

Removing the stochastic term from Eq. (15) yields the deterministic probability flow ODE:

$$\frac{dz_t}{dt} = -\mu(t) z_t - \sigma(t)^2 \nabla_z \log p_t(z_t). \tag{16}$$

When the forward process is chosen to be variance-preserving ($\mu(t) = 0$), the first term vanishes and the flow reduces to:

$$\frac{dz_t}{dt} = -\sigma(t)^2 \nabla_z \log p_t(z_t). \tag{17}$$

We now establish the equivalence to the AR flow in Eq. (13). Define a smooth, strictly increasing reparameterization $s = \phi(t)$ with $\phi(0) = 0$ and $\phi(1) = 1$, given by

$$s(t) \triangleq \frac{\int_0^t \sigma(\tau)^2 \, d\tau}{\int_0^1 \sigma(\tau')^2 \, d\tau'}. \tag{18}$$

Applying the chain rule to Eq. (17) gives

$$\begin{aligned} \frac{dz_s}{ds} &= \frac{dz_t}{dt} \cdot \frac{dt}{ds} \\ &= \left[-\sigma(t)^2 \nabla_z \log p_t(z_t)\right] \frac{C}{\sigma(t)^2}, \end{aligned} \tag{19}$$

where $C = \int_0^1 \sigma(\tau')^2 \, d\tau'$ is a positive constant. Simplifying, we obtain

$$\frac{dz_s}{ds} = -C \, \nabla_z \log p_s(z_s). \tag{20}$$

Eq. (20) has the same form as the AR flow in Eq. (13), with the identification

$$F_\theta(x_s, s) \equiv -C \, \nabla_x \log p_s(x_s). \tag{21}$$

That is, the deterministic AR evolution is a scaled version of the probability flow ODE for a score-based model, when both are parameterized over the same continuous-time latent trajectory. The AR model can thus be viewed as following the same score field as the diffusion model, but with a different time parametrization and without stochasticity.

Finally, under the standard assumptions that $\sigma(t)$ is continuous and strictly positive, and that p_t admits a differentiable score function $\nabla_z \log p_t(z)$, the mapping in Eq. (18) is smooth and invertible, ensuring the equivalence holds globally over $t \in [0, 1]$. Therefore, autoregressive generation and deterministic diffusion flows are provably two discretizations of a shared continuous-time generative process, completing the proof.

C Case Studies

We present qualitative outputs from **Evo-8B Instruct** to illustrate its hybrid autoregressive–diffusion generation capabilities. The examples demonstrate (1) coherent and fluent text generation in a mixed-step decoding regime, (2) multi-turn dialogue with consistent context retention, and (3) adaptive reasoning that leverages fine-grained token refinement.

Mathematical Reasoning. In Table 3, Evo correctly solves a multi-step word problem by first drafting an initial reasoning chain and then selectively refining uncertain tokens in later diffusion-style steps.

Table 3: Mathematical reasoning with adaptive refinement.

User: Lily can run 12 kilometers per hour for 4 hours. After that, she runs 6 kilometers per hour for another 3 hours. How many kilometers does she run in total? **Evo:** First, at 12 km/h for 4 hours, Lily covers 48 kilometers. Then, at 6 km/h for 3 hours, she covers 18 kilometers. Adding them gives 66 kilometers in total.

Multi-Turn Dialogue. Table 4 shows Evo engaging in a multilingual multi-turn conversation. The model maintains dialogue coherence across turns, translating between languages, and composing text under lexical constraints.

Table 4: Multi-turn dialogue with translation and constrained generation.

User: Could you share the first two lines of the poem "The Road Not Taken"? **Evo:** "Two roads diverged in a yellow wood, And sorry I could not travel both." **User:** Translate into German. **Evo:** "Zwei Wege trennten sich im gelben Wald, und leider konnte ich nicht beide gehen." **User:** Now write a short poem about life choices, each sentence starting with "C". **Evo:** Choices shape the path we walk. Courage guides us through the unknown. Curiosity opens hidden doors. Commitment carries us forward. Clarity shows the way home.

Step-Adaptive Story Generation. Evo's hybrid decoding allows for global planning via coarse AR passes followed by localized diffusion refinements. Table 5 shows a short science-fiction story where key sentences are refined for narrative flow.

Table 5: Step-adaptive story generation. Italics indicate refined sentences.

User: Write a short science-fiction story about a time-traveling botanist. **Evo:** Dr. Amina travelled to the year 3050, carrying seeds from the past. She planted them in Martian soil, curious to see what might grow. *Under the twin moons, the seedlings shimmered with an unearthly glow.* The Martian settlers gathered to witness the strange blossoms. *In that moment, Amina realized she had brought not just plants, but hope, across centuries.*

Multi-Step Reasoning. Evo decomposes complex tasks into intermediate steps, benefiting from its ability to revise earlier parts of the reasoning chain without regenerating the entire sequence.

Table 6: Mathematical reasoning with token refinement.

User: A tank is filled at 15 liters per minute for 8 minutes, then at 10 liters per minute for 12 minutes. How many liters are in the tank? **Evo:** First stage: $15 \times 8 = 120$ liters. Second stage: $10 \times 12 = 120$ liters. Adding gives 240 liters in total.

Multilingual Dialogue. Evo smoothly transitions across languages while retaining context, demonstrating both translation and constrained text generation.

Table 7: Multi-turn multilingual dialogue.

User: What is the proverb "A stitch in time saves nine" in German? **Evo:** German: "Wer rechtzeitig handelt, erspart sich viel Mühe." **User:** Write a haiku in English using the word "time". **Evo:** Time flows like a stream, Leaves drift in a quiet dream, Dawn breaks, gold and clean.

Code Completion. Evo's AR steps quickly produce a syntactically correct draft, while refinement passes fill in missing logic.

Table 8: Python function generation.

User: Write a Python function to check if a number is prime.
Evo:
```
def is_prime(n):
if n < 2: return False
for i in range(2, int(n**0.5)+1):
if n % i == 0: return False
return True
```

Creative Storytelling. Hybrid decoding enables Evo to generate coherent narratives and then improve phrasing without losing global structure.

Table 9: Short story with refinement. Italics indicate refined sentences.

User: Write a short story about a lost map in the desert.
Evo: The wind buried the caravan's trail in minutes. Karim unfolded the old map, only to find half the ink faded. *The sun blazed overhead, turning sand into a sea of gold.* He followed the faint lines until they ended at a lone acacia tree. *Beneath its roots, a small tin box held the real map, untouched by time.*

Reversal Reasoning. Unlike pure AR models, Evo treats tokens without strict left-to-right bias, allowing it to recover missing preceding context.

Table 10: Poem reversal completion.

User: Given the line "And sorry I could not travel both", write the preceding line from Robert Frost's "The Road Not Taken".
Evo: "Two roads diverged in a yellow wood."

Across all domains, Evo benefits from its hybrid AR–diffusion decoding in two key ways. First, the AR component ensures rapid generation of a globally coherent structure, making it competitive in speed with pure autoregressive LLMs. Second, the diffusion-style refinement selectively updates uncertain tokens, enabling corrections without regenerating the entire sequence. This is particularly evident in reasoning (Tables 6 and 10), where local numerical or contextual corrections are made after the global plan is laid out. In multilingual dialogue (Table 7), the retention of long-range context across turns is aided by the bidirectional attention in refinement steps, which allows earlier utterances to influence later token predictions even in different languages. For creative writing (Table 9), refinement passes improve thematic consistency and stylistic flow, showing how Evo can balance efficiency and quality. Finally, in reversal reasoning (Table 10), Evo's non-directional refinement offers an inherent advantage over strictly causal models, which often struggle when the missing span is at the start of the context.